

PANDAS

Complete Syllabus & Study Guide



BROWSEJOBS

Module 1: Introduction to Pandas & Data Structures

1.1 What is Pandas?

Pandas is a powerful Python library used for **data manipulation and analysis**. It provides data structures like **Series** and **DataFrame** that make it easy to clean, transform, and analyze structured data.

In data engineering, Pandas is the go-to tool for processing CSV files, handling missing data, removing duplicates, fixing data types, and preparing data for downstream analytics.

1.2 Why Pandas is Essential for Data Engineers

The four pillars of data cleaning that Pandas addresses:

Problem	Description	Pandas Solution
Missing Data	Rows with blank or NaN values	<code>fillna()</code> , <code>dropna()</code>
Duplicate Data	Identical rows appearing multiple times	<code>drop_duplicates()</code>
Wrong Data Type	Numbers stored as strings, dates as text	<code>astype()</code> , <code>to_datetime()</code>
Wrong/Invalid Data	Outliers, typos, impossible values	<code>loc[]</code> , <code>conditional logic</code>

📄 **Real-World Scenario:** You receive a daily CSV from the finance team at midnight. The file has 1 million rows with missing revenue values, duplicate entries, and dates in inconsistent formats. Pandas lets you write a script that automatically cleans all of this before loading it into a data warehouse.

1.3 Arrays vs Lists & 1.4 Pandas Series

Arrays vs Lists in Python

Before understanding Pandas, it helps to know the difference between arrays and lists in Python:

Feature	Array	List
Data Types	Homogeneous (same type only)	Heterogeneous (any type)
Example	<code>x = [1, 2, 3, 4, 5]</code>	<code>x = [1, 2, 'a', True]</code>
Performance	Faster for numeric operations	Flexible but slower
Use Case	Mathematical computations	General-purpose storage

Pandas Series

A **Series** is a one-dimensional labeled array. Think of it as a single column in a spreadsheet or database table. Each element has an index (label) and a value.

```
import pandas as pd

a = [1, 7, 2]
myvar = pd.Series(a)
# Output:
# 0    1
# 1    7
# 2    2
# dtype: int64

# Series with Custom Index:
myvar = pd.Series(a, index=["x", "y", "z"])
# x    1
# y    7
# z    2
# Access by label:
myvar['y'] # Returns 7
```

 **Real-World Scenario:** Imagine tracking daily website traffic. A Series with date labels lets you quickly look up traffic for any specific day: `traffic['2025-01-15']` returns that day's visitor count.



1.5 Key Takeaways

Foundation

Pandas is the foundational framework for data cleaning in Python.

Series = 1D

Series is a 1D labeled array (like a column).

Index Access

The index lets you access data by label name, not just position.

Building Block

Understanding Series is the building block for working with DataFrames.

Module 2: DataFrames – Creation & Exploration

2.1 What is a DataFrame?

A **DataFrame** is a 2-dimensional labeled data structure with columns of potentially different types. Think of it as a spreadsheet, SQL table, or a dictionary of Series objects. It is the **primary Pandas object** you will work with in data engineering.

Creating a DataFrame from a Dictionary

```
import pandas as pd

data = {
    "calories": [420, 10, 100, 50],
    "duration": [50, 5, 50, 65]
}

df = pd.DataFrame(data)
print(df)
#   calories  duration
# 0     420         50
# 1       10          5
# 2     100         50
# 3       50         65
```

2.2 Exploring DataFrames

Method	Purpose
<code>df.head()</code>	View top 5 rows (quick schema check)
<code>df.tail()</code>	View bottom 5 rows
<code>df.info()</code>	Column names, data types, non-null counts
<code>df.shape</code>	Number of rows and columns
<code>df.describe()</code>	Statistical summary (mean, std, etc.)

2.2 (cont.) Using `df.info()` – The Most Important Method

`df.info()` is your **first stop after loading any data**. It reveals missing values, data types, and memory usage:

```
df.info()
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 32 entries, 0 to 31
# Data columns (total 5 columns):
# #   Column   Non-Null Count  Dtype
# 0  Duration  32 non-null    int64
# 1  Date      31 non-null    object  <-- 1 missing!
# 2  Pulse     32 non-null    int64
# 3  Maxpulse  32 non-null    int64
# 4  Calories  30 non-null    float64  <-- 2 missing!
```

📌 **Real-World Scenario:** When a finance team sends you a CSV with 1 million rows and 30 columns, running `df.info()` immediately tells you which columns have missing data, which have wrong data types (e.g., Date showing as 'object' instead of datetime), and the total memory footprint.

2.3 Key Takeaways

2D Structure

DataFrames are 2D structures (rows + columns) and the core object for all data operations.

Always Run `df.info()` First

Always run `df.info()` first to understand your data's health before starting any transformation.

Module 3: Indexing & Selection (loc vs iloc)

3.1 Understanding loc and iloc

Accessing specific rows and columns is fundamental. Pandas provides two main indexers:

Feature	loc (Label-based)	iloc (Integer-based)
Access Method	Uses row/column names (labels)	Uses row/column positions (integers)
Syntax	<code>df.loc[row_label, col_name]</code>	<code>df.iloc[row_index, col_index]</code>
Example	<code>df.loc[0, 'calories']</code>	<code>df.iloc[0, 1]</code>
Use When	You know column names	You know position numbers

Examples

```
import pandas as pd
df = pd.read_csv('customers.csv')

# Using loc - access by label name
df.loc[0, 'calories']
# Gets value at row 0, column 'calories'

# Using iloc - access by integer position
df.iloc[0, 1]
# Gets value at row 0, column position 1

df.iloc[2, 1]
# Gets value at row 2, column position 1
```

When to Use Which?

Use **loc** when you know the column names (most common in real projects). Use **iloc** when processing data programmatically by position, such as iterating through columns by index number.

 **Real-World Scenario:** In an ETL pipeline, you receive a customer CSV where you need to extract the 'email' column by name (`loc`) for validation. But sometimes you get files with inconsistent column names, so you fall back to `iloc` to grab the 3rd column regardless of its header name.

Module 4: Reading Data from External Sources



4.1 Reading CSV Files

CSV (Comma Separated Values) is the most common file format you will encounter in data engineering. Pandas makes reading CSVs effortless:

```
import pandas as pd
df = pd.read_csv('/path/to/data.csv')
df.head() # Always check first rows
```



4.2 Reading JSON Files

JSON (JavaScript Object Notation) is the standard format for API responses and system-to-system data exchange:

```
import pandas as pd
df = pd.read_json('/path/to/data.json')
df.head()
```



4.3 Reading Parquet Files

Parquet is a columnar storage format optimized for big data. It is compressed, fast, and widely used in data engineering:

```
import pandas as pd
df = pd.read_parquet('/path/to/data.parquet')
df.head()
```

4.4 File Format Comparison

Format	Type	Best For	Human Readable?
CSV	Row-based, text	Small datasets, sharing	Yes
JSON	Semi-structured	APIs, nested data	Yes
Parquet	Columnar, binary	Big data, analytics	No
Avro	Row-based, binary	Streaming, write-heavy	No
ORC	Columnar, binary	Hive/Hadoop analytics	No
Excel	Row-based	Business reports	Yes (in Excel)



Module 5: Handling Missing Data (NaN / Null Values)

5.1 Why Missing Data Matters

Missing data is the most common data quality issue. It can cause errors in calculations, break machine learning models, and produce incorrect reports. Pandas represents missing data as **NaN (Not a Number)**.

5.2 Detecting Missing Data

```
df.info()      # Shows non-null count per column
df.isnull().sum() # Count of nulls per column
```

5.3 Strategy 1: Drop Missing Values (dropna)

Use when missing data is a small percentage and dropping rows won't significantly reduce your dataset:

```
# Creates a NEW dataframe without nulls
newdf = df.dropna()

# Modifies the ORIGINAL dataframe
df.dropna(inplace=True)
```

❏ **Important:** `dropna()` removes the ENTIRE row if ANY column has a null value. Use with caution on datasets where only specific columns have missing values.

5.4 Strategy 2: Fill Missing Values (fillna)

```
# Option A: Fill with a Fixed Value
newdf = df.fillna(130)
df['Calories'] = df['Calories'].fillna(130)

# Option B: Fill with Mean (Average)
x = df['Calories'].mean() # e.g. 304.68
df['Calories'] = df['Calories'].fillna(x)

# Option C: Fill with Median (Middle Value)
x = df['Calories'].median()
df['Calories'] = df['Calories'].fillna(x)

# Option D: Fill with Mode (Most Frequent)
x = df['Calories'].mode()[0] # Returns 300.0
df['Calories'] = df['Calories'].fillna(x)
```

5.5–5.6 Filling by Column Type & Mean vs Median vs Mode

5.5 Filling Different Column Types Differently

In real projects, you apply different strategies based on column type:

```
# Numerical columns: fill with mean
df['Revenue'] = df['Revenue'].fillna(df['Revenue'].mean())
df['Expenses'] = df['Expenses'].fillna(df['Expenses'].mean())

# Date columns: fill with a default date
df['Date'] = df['Date'].fillna('1900-01-01')

# String columns: fill with 'unknown'
df['Department'] = df['Department'].fillna('unknown')
```

5.6 Mean vs Median vs Mode – When to Use

Mean

Best For: Normal distribution, no outliers

Weakness: Skewed by extreme values

Median

Best For: Skewed data with outliers

Weakness: Ignores distribution shape

Mode

Best For: Categorical data, discrete values

Weakness: May not represent center

📌 **Real-World Scenario:** A hospital's patient records have missing 'Age' values. Using mean (45.2) works well for normally distributed ages. But if the 'Income' column has a few billionaires, median is better because the mean would be artificially inflated.



Module 6: Removing Duplicates & Module 7: Fixing Wrong Data

6.1 Why Duplicates Occur

Duplicates happen when data is ingested multiple times, systems retry failed uploads, or the same record exists in multiple source files. They **inflate counts, distort averages, and cause double-processing**.

6.2 Detecting and Removing Duplicates

```
# Check for duplicates
df.duplicated().sum() # Count of duplicate rows

# Remove duplicates (keeps first occurrence)
df.drop_duplicates(inplace=True)

# Verify removal
df.info() # Check new row count
```

The exercise data had rows 11 and 12 as identical (same Date '2020/12/12', same values). After `drop_duplicates()`, row count dropped from 32 to 31.

 **Real-World Scenario:** An e-commerce platform's order system retries failed API calls, causing some orders to appear twice. Running `drop_duplicates()` on the `order_id` column ensures each order is counted only once for revenue reporting.

7.1 Identifying Wrong Data

Wrong data includes outliers, impossible values, or data that violates business rules. For example, a workout duration of **450 minutes** is clearly an error when all other values are between 30–75 minutes.

7.2 Fixing Outliers with Conditional Logic

```
# Cap Duration values above 120 to 120
for i in df.index:
    if df.loc[i, 'Duration'] > 120:
        df.loc[i, 'Duration'] = 120

# Row 7 had Duration=450, now corrected to 120
```

7.3 Data Validation Checklist

- Naming Convention: File follows expected pattern: `finance_01012026.csv`
- File Type: Received .csv matches expected format
- File Size: Size is within expected range (not empty or unusually large)
- Row Count: Actual rows vs expected rows
- Schema Validation: Expected number of columns matches actual
- Data Type Validation: Expected dtypes match actual dtypes
- Missing Data Count: Count of nulls per column within threshold
- Duplicate Validation: No unexpected duplicate rows
- Wrong Data Check: Values within acceptable business ranges

Module 8: Data Type Conversions

8.1 Why Data Types Matter

Incorrect data types prevent operations like date arithmetic, numerical aggregations, and sorting. The most common issue is **dates stored as strings (object type)**.

8.2 Converting Date Strings to DateTime

The exercise dataset had dates in mixed formats ('2020/12/01', '26-12-2020', NaN). Here is the cleaning process:

```
# Step 1: Fill missing dates with a default
df['Date'] = df['Date'].fillna("1900/01/01")

# Step 2: Convert to datetime (handles mixed formats)
df['Date'] = pd.to_datetime(df['Date'])

# Step 3: Standardize format for output
df['Date'] = df['Date'].dt.strftime('%d-%m-%Y')
```

8.3 Common Data Type Operations

Operation	Code	Use Case
String to Integer	df['col'].astype(int)	Age stored as '16' text
String to Float	df['col'].astype(float)	Price stored as text
String to DateTime	pd.to_datetime(df['col'])	Date analysis
DateTime to String	df['col'].dt.strftime('%Y-%m-%d')	Standard output format
Check all types	df.dtypes	Quick overview

📌 **Real-World Scenario:** A retail company's sales data has the 'sale_date' column as text (object type). You cannot filter by date ranges or calculate monthly totals until you convert it to datetime. After conversion, queries like `df[df['sale_date'] > '2025-01-01']` work correctly.

Module 9: Merging & Joining DataFrames

9.1 Why Joins Matter

In real projects, data is spread across multiple tables (students + marks, customers + orders, employees + departments). Joining lets you **combine them into a single view**.

9.2 Types of Joins

Inner Join

Only matching rows from both tables

SQL: INNER JOIN

Left Join

All rows from left + matching from right

SQL: LEFT JOIN

Right Join

All rows from right + matching from left

SQL: RIGHT JOIN

Outer Join

All rows from both tables

SQL: FULL OUTER JOIN

9.3 Pandas Merge Syntax

```
df1 = pd.DataFrame({'id': [1,2,3,4],
                    'Name': ['krish','ajay','adveer','anuj']})
df2 = pd.DataFrame({'id': [3,4,5],
                    'age': [25,28,22]})

# Inner Join
output = pd.merge(df1, df2, on='id', how='inner')
# Result: id 3 (adveer,25) and id 4 (anuj,28)

# Left Join
output = pd.merge(df1, df2, on='id', how='left')
# All 4 from df1, age=NaN for ids 1,2

# Right Join
output = pd.merge(df1, df2, on='id', how='right')
# All 3 from df2, Name=NaN for id 5

# Full Outer Join
output = pd.merge(df1, df2, on='id', how='outer')
# All 5 unique ids, NaN where no match
```



9.4 Joining on Multiple Columns

```
output = pd.merge(df1, df2,  
on=['id', 'city'], how='inner')
```

9.5 Chaining Multiple Joins

When you have 3+ tables, chain merges sequentially:

```
df1 = pd.DataFrame({'id': [1,2,3],  
                    'Name': ['john','Anna','Peter']})  
df2 = pd.DataFrame({'id': [3,4,5],  
                    'age': [25,28,22]})  
df3 = pd.DataFrame({'id': [1,2,3],  
                    'country': ['USA','UK','India']})  
  
step1 = pd.merge(df1, df2, on='id', how='inner')  
final = pd.merge(step1, df3, on='id', how='inner')  
# Result: id=3, Peter, age=25, India
```

- 📌 **Real-World Scenario:** An HR system stores employee info in one table, salary data in another, and department details in a third. To build a comprehensive employee report, you inner join all three on `employee_id` to get name + salary + department in one DataFrame.



Module 10: Pivot Tables

10.1 What is a Pivot Table?

A pivot table **reshapes data** by turning unique values from one column into multiple columns. It transforms 'long' format data into 'wide' format for easier comparison and reporting.

10.2 Basic Pivot

```
import pandas as pd

df = pd.DataFrame({
    'date': ['2021-01-01', '2021-01-01',
            '2021-01-02', '2021-01-02'],
    'city': ['newyork', 'los angeles',
            'newyork', 'los angeles'],
    'temperature': [32, 75, 34, 77]
})

newdf = df.pivot(
    index='date',
    columns='city',
    values='temperature'
)

# Result:
# city    los angeles  newyork
# 2021-01-01         75      32
# 2021-01-02         77      34
```

10.3 Multi-Value Pivot

You can pivot multiple value columns at once:

```
newdf = df.pivot(
    index='Date',
    columns='City',
    values=['Temperature',
            'Humidity',
            'Wind Speed']
)

# Creates a multi-level column header
```

📌 **Real-World Scenario:** A sales team tracks revenue by product and region in a long table (product, region, revenue). Pivoting with region as columns creates a comparison table showing each product's revenue across all regions side by side – perfect for executive dashboards.



Module 11: JSON Normalization

11.1 Handling Nested JSON

APIs often return deeply nested JSON data. `json_normalize()` **flattens nested structures** into a flat DataFrame.

Flat JSON

```
import pandas as pd

data = {
    'id': 123,
    'name': 'krish',
    'address': {
        'street': '123 main st',
        'city': 'bangalore',
        'state': 'karnataka'
    }
}

df = pd.json_normalize(data)
# Columns: id, name, address.street,
#         address.city, address.state
```

📌 **Real-World Scenario:** A payment gateway API returns transaction data with nested 'customer' and 'items[]' fields. Using `json_normalize` with `record_path='items'` and `meta=['customer.id', 'transaction_date']` flattens it into one row per item with customer info preserved.

Nested JSON with Arrays

```
data = [
    {'id': '456', 'name': 'ajay',
     'orders': [
         {'order_id': 'A1', 'item': 'laptop',
          'price': 1000},
         {'order_id': 'A2', 'item': 'mobile',
          'price': 500}
     ]},
    {'id': '789', 'name': 'krish',
     'orders': [
         {'order_id': 'B1', 'item': 'laptop',
          'price': 1000},
         {'order_id': 'B2', 'item': 'mobile',
          'price': 500}
     ]}
]
```

```
df = pd.json_normalize(data,
    record_path='orders',
    meta=['id', 'name'])
# Columns: order_id, item, price, id, name
# Each order becomes its own row
# with parent info attached
```

Module 12: File Formats & Parquet Deep Dive

12.1 Why Parquet?

Feature	Parquet	CSV
Storage	Columnar (reads only needed columns)	Row-based (reads entire rows)
Compression	Built-in (Snappy, Gzip)	None by default
Size	5–10x smaller than CSV	Full text size
Speed	Optimized for analytical queries	Slower for large files
Schema	Embedded metadata (types preserved)	No schema (everything is text)
Human Readable	No (binary format)	Yes (text file)

12.2 Converting Between Formats

```
import pandas as pd

# CSV to Parquet
df = pd.read_csv('data.csv')
df.to_parquet('data.parquet')

# Parquet to CSV
df = pd.read_parquet('data.parquet')
df.to_csv('data.csv', index=False)

# Parquet round-trip preserves data types!
```

📌 **Real-World Scenario:** Your raw CSV file is 130MB. After converting to Parquet, it shrinks to ~15MB. Query speed improves 10x because Parquet only reads the columns your query needs instead of scanning every row.

12.3 File Format Quick Reference

Format	Storage	Best Use Case
CSV	Row-based, text	Small data, human readable
JSON	Semi-structured	APIs, nested data
Parquet	Columnar	Analytics, ML, big data
Avro	Row-based	Streaming, schema evolution
ORC	Columnar	Hive/Hadoop analytics



Module 13: Processing Large Files with Chunking

13.1 The Memory Problem

Loading a **10GB CSV into memory crashes most machines**. Chunking reads the file in small pieces (e.g., 100,000 rows at a time), processes each piece, and writes results incrementally.

13.2 Chunked Processing Pattern

```
import pandas as pd

chunk_size = 100000
first_chunk = True

for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
    # Transform each chunk
    chunk['upper_city'] = chunk['city'].str.upper()

    # Write to output (append mode after first chunk)
    chunk.to_csv('output.csv', mode='a', index=False,
                header=first_chunk)
    first_chunk = False # Only write header once
```

Append Mode

The 'a' mode appends to the file instead of overwriting.

Header Flag

The header flag ensures column names appear only once at the top.

Any File Size

This pattern can process files of ANY size with constant memory usage.

- ❏ **Real-World Scenario:** A telecom company generates 50 million call records daily (8GB CSV). Your ETL script processes it in chunks of 500,000 rows, cleaning phone numbers and calculating call durations, writing clean data to Parquet without ever loading the full file.



Module 14: Merging Multiple Files with Glob

14.1 The Problem

Data often arrives as **multiple files** (one per day, region, or department). You need to combine them into a single DataFrame for analysis.

14.2 Using Glob + pd.concat

```
import pandas as pd
import glob

# List all CSV files in a folder
files = glob.glob('/path/to/folder/*.csv')
# Returns: ['1.csv', '2.csv', '3.csv']

# Read and merge all files into one
# DataFrame
merged_df = pd.concat(
    [pd.read_csv(file) for file in files],
    ignore_index=True
    # Reset index for the new combined df
)

# Save the merged result
merged_df.to_csv('merged_output.csv',
                 index=False)
```

Why This Pattern Works

1 glob.glob()

Finds all matching files in a directory using a wildcard pattern like `*.csv`.

2 List Comprehension

Reads each file into a DataFrame in a single line using a Python list comprehension.

3 pd.concat()

Stacks all DataFrames vertically into one master dataset with `ignore_index=True` to reset the row index.



Real-World Scenario: A logistics company receives one CSV per warehouse per day. At month-end, you need to combine 30 files x 10 warehouses = 300 CSVs into one master dataset for inventory reporting. Glob + concat handles this in 3 lines of code.

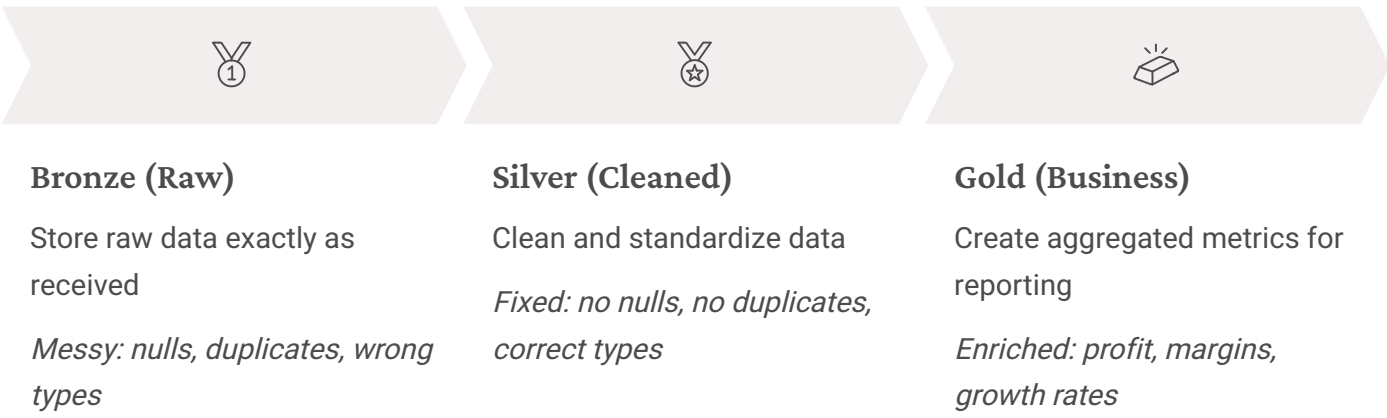
Module 15: End-to-End ETL Project

(Medallion Architecture)

15.1 Project Overview

This module ties everything together into a real-world **ETL (Extract, Transform, Load) pipeline** using the Medallion Architecture pattern used by companies like Databricks.

15.2 Medallion Architecture



Scenario: A CSV file arrives from the finance team every morning at 12 AM CST. You must clean it and produce a reporting-ready Parquet file.

15.3 The Finance Data Pipeline – Step by Step

Step 1: Extract (Bronze Layer)

```
import pandas as pd
df = pd.read_csv('/path/to/finance_data.csv')
df.info()
# Inspect: 105 rows, 13 columns,
# multiple null columns
```

Step 2: Transform (Silver Layer)

```
# 1. Remove duplicates
df.drop_duplicates(inplace=True)

# 2. Fill nulls by data type
# Numerical: fill with mean
for col in ['Revenue', 'Expenses',
            'COGS', 'Operating_Expenses']:
    df[col] = df[col].fillna(df[col].mean())

# Date: fill with default
df['Date'] = df['Date'].fillna('1900-01-01')

# String: fill with 'unknown'
df['Department'] = df['Department'].fillna('unknown')
```

Step 3: Enrich (Gold Layer)

```
df['total_revenue'] = df['Revenue']
df['total_expense'] = df['Expenses']
df['profit'] = df['Revenue'] - df['Expenses']
df['gross_margin'] = (df['Revenue'] - df['COGS']) / df['Revenue']
df['operating_income'] = df['Revenue'] - df['Operating_Expenses']
df['EBITDA'] = df['operating_income'] + df['Depreciation'] + df['Amortization']
df['accounts_receivables'] = df['AR']
df['accounts_payables'] = df['AP']
df['year_over_year_growth'] = df['Revenue'] * 0.05
```

Step 4: Load

```
# Save as Parquet for downstream analytics
df.to_parquet('/output/finance_gold.parquet')

# Also save as CSV for non-technical stakeholders
df.to_csv('/output/finance_gold.csv', index=False)
```

15.4 Business Metrics & Career Tip

Business Metrics Explained

Metric	Formula	What It Tells You
Profit	Revenue - Expenses	How much money the company keeps
Gross Margin	(Revenue - COGS) / Revenue	Efficiency of production
Operating Income	Revenue - Operating Expenses	Profitability from core operations
EBITDA	Operating Income + Depreciation + Amortization	Cash-generating ability
YoY Growth	Revenue × Growth Rate	Revenue trend over time

 Career Tip

This exact ETL pattern (**read CSV → clean → enrich → write Parquet**) is what data engineers do daily at companies like **Amazon, Google, and Goldman Sachs**. Master this flow, and you can handle any data pipeline interview question.